

Linked List Management System

DSA Mini Project - 2nd Year College Level

1. Project Overview

Create a menu-driven console application for practicing Linked List data structures. The student may choose **either C++ or Java**. The project must contain both a **Singly Linked List** and a **Doubly Linked List**, with the required operations implemented in the selected language.

2. Language Choice - C++ or Java

The student can choose any one language: **C++ OR Java**. The data-structure logic, menu options, testing requirements, and project scope remain the same.

C++	Java
cout / cin	System.out / Scanner
Pointers and references	Object references
new / delete	new / garbage collection
struct/class	class

3. Main Application Menu

```
=====
LINKED LIST MANAGEMENT SYSTEM
=====

1. Singly Linked List
2. Doubly Linked List
3. Compare Singly vs Doubly Linked List
4. Exit
```

Enter your choice:

4. Singly Linked List - Complete Menu

```
***** Operation on Singly Linked List *****
```

```
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Insert After a Node
5. Insert Before a Node

6. Delete from Beginning
7. Delete from End
8. Delete from Position
9. Delete by Value

10. Search
11. Update
12. Display
13. Count Nodes
14. Find Maximum
15. Find Minimum
16. Find Middle Element
17. Reverse
18. Sort
19. Remove Duplicates
20. Exit
```

Enter the choice:-

5. Singly Linked List - Operation Description

Insert at Beginning - Add a new node before head. Example: 10 -> 20 -> 30 becomes 5 -> 10 -> 20 -> 30. $O(1)$.

Insert at End - Traverse to the last node and attach the new node. $O(n)$ without a tail pointer/reference.

Insert at Position - Insert at a 1-based position. Handle position 1, middle, last-valid position, and invalid positions.

Insert After a Node - Find the first matching value and insert the new node immediately after it.

Insert Before a Node - Find the first matching value and insert the new node immediately before it.

Delete from Beginning - Remove head and move head to the next node. $O(1)$.

Delete from End - Remove the last node and correctly update the previous node.

Delete from Position - Delete the node at a given 1-based position and preserve all links.

Delete by Value - Delete the first node containing a target value; report when not found.

Search - Search for a value and display its position if found.

Update - Find a value and replace its data with a new value.

Display - Traverse from head to NULL and print the linked list.

Count Nodes - Traverse and count all nodes.

Find Maximum - Traverse and return the largest value.

Find Minimum - Traverse and return the smallest value.

Find Middle Element - Use slow/fast pointers or references. Define the rule for even-length lists.

Reverse - Reverse next links using previous, current, and next. $O(n)$, extra space $O(1)$.

Sort - Sort ascending. Beginner: Bubble Sort by swapping data, $O(n^2)$. Advanced: Merge Sort.

Remove Duplicates - Remove repeated values while maintaining valid links.

6. Doubly Linked List - Complete Menu

***** Operation on Doubly Linked List *****

1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Insert After a Node
5. Insert Before a Node

6. Delete from Beginning
7. Delete from End
8. Delete from Position
9. Delete by Value

10. Search
11. Update
12. Display Forward
13. Display Backward
14. Count Nodes
15. Find Maximum
16. Find Minimum
17. Find Middle Element
18. Reverse
19. Sort
20. Remove Duplicates
21. Exit

Enter the choice:-

7. Doubly Linked List - Operation Description

Insert at Beginning - Insert before head and update both prev and next links.

Insert at End - Attach after the tail and update the new node's prev link.

Insert at Position - Insert at a 1-based position and maintain both directions.

Insert After a Node - Insert after the first matching node and reconnect neighboring links.

Insert Before a Node - Insert before the first matching node and reconnect neighboring links.

Delete from Beginning - Remove head and update the new head's prev link.

Delete from End - Remove tail and update the previous node's next link.

Delete from Position - Delete a node at a specified position while preserving both links.

Delete by Value - Delete the first matching node.

Search - Find a value and report its position.

Update - Replace the data of a matching node.

Display Forward - Traverse using next links.

Display Backward - Start at the tail and traverse using prev links.

Count Nodes - Count all nodes.

Find Maximum - Find the largest value.

Find Minimum - Find the smallest value.

Find Middle Element - Find the middle using slow/fast references or a length-based method.

Reverse - Reverse both prev and next links and update head/tail correctly.

Sort - Sort ascending. Beginner: Bubble Sort; advanced: Merge Sort.

Remove Duplicates - Remove repeated values while maintaining correct prev/next links.

8. Example Output

Singly Linked List

Before:

40 -> 10 -> 30 -> 20 -> 50 -> NULL

After Sort:

10 -> 20 -> 30 -> 40 -> 50 -> NULL

After Reverse:

50 -> 40 -> 30 -> 20 -> 10 -> NULL

Doubly Linked List

Forward:

NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL

Backward:

NULL <- 40 <-> 30 <-> 20 <-> 10 -> NULL

9. Complexity Expectations

Operation	Typical Complexity
Insert at Beginning	O(1)
Insert at End	O(n) without tail
Insert at Position	O(n)
Delete from Beginning	O(1)
Delete from End	O(n) in a singly list
Search / Display / Count	O(n)
Find Min / Max	O(n)
Find Middle	O(n)
Reverse	O(n), extra space O(1)
Bubble Sort	O(n ²)
Simple Remove Duplicates	O(n ²)

10. Edge Cases to Test

- Empty linked list.
- One-node list.
- Insert/delete at position 1.
- Insert at the last valid position.
- Invalid or out-of-range position.
- Delete a value that does not exist.
- Duplicate values.
- Zero and negative values.
- Sort an empty or single-node list.
- Reverse an empty or single-node list.

11. Recommended Development Order

Level 1: Node class/structure, create list, display, insert at beginning and end.

Level 2: Delete from beginning/end, search, count.

Level 3: Insert/delete at position, before/after, delete by value.

Level 4: Update, maximum, minimum, middle.

Level 5: Reverse, sort, remove duplicates.

Level 6: Implement the Doubly Linked List and compare pointer/reference management.

12. GitHub Submission Requirement

The project must be submitted through GitHub. The student should create a GitHub repository and upload the complete source code for the selected language (C++ or Java). The repository link should be submitted as the final project submission.

Recommended GitHub Repository Structure

```
Linked-List-Management-System/
|
|-- README.md
|-- src/
|   |-- SinglyLinkedList.cpp / SinglyLinkedList.java
|   |-- DoublyLinkedList.cpp / DoublyLinkedList.java
|   |-- Main.cpp / Main.java
|
|-- sample-output/
|   |-- output.txt
|
|-- docs/
|   |-- project-report.pdf
|
`-- .gitignore
```

GitHub Submission Checklist

- **README.md:** Project title, objective, selected language, features, how to run, and sample output.
- **Source code:** Complete working code for both Singly and Doubly Linked Lists.
- **Sample output:** Include screenshots or a text output file showing important operations.
- **Project report:** Optional/required according to the instructor; include complexity and design explanation.
- **Repository link:** Submit the public GitHub repository URL to the instructor.

Suggested README Sections

```
# Linked List Management System

## Objective
## Language Used: C++ or Java
## Features
## Singly Linked List
## Doubly Linked List
## Operations
## How to Run
## Sample Output
## Time Complexity
## Author
```

13. Viva / Evaluation Questions

1. What is a linked list?
2. What is the difference between an array and a linked list?
3. What is the purpose of head?

4. What is the purpose of next?
5. Why does a doubly linked list need prev and next?
6. Why is insertion at the beginning $O(1)$?
7. Why is searching $O(n)$?
8. How do you reverse a singly linked list?
9. How do slow and fast pointers find the middle?
10. How do you safely delete a node?
11. What changes when implementing the project in Java instead of C++?
12. Why does Java not require explicit delete?
13. Why is Bubble Sort $O(n^2)$?
14. How can Merge Sort be used for linked lists?
15. What edge cases must be handled?

14. Final Submission Requirements

- Choose **one language only: C++ or Java**.
- Implement both Singly and Doubly Linked Lists.
- Implement all required operations as separate functions/methods.
- Do not replace the linked list implementation with an array or built-in list collection.
- Test all important edge cases.
- Upload the complete project to GitHub.
- Include a clear README and sample output.
- Submit the GitHub repository link.